

FEWLLM Data Agent

KDD Cup 2026 DataAgent-Bench — Course Project

MA Ruixin, LIN Yipeng, XIAO Yao

Society Hub

The Hong Kong University of Science and Technology (Guangzhou)

Spring 2026



① Problem & Architecture

Task, scoring rule, four-layer design

② Agent Loop & Routing

TaskCompiler, Controller, Tool actions

③ Risk Gating (Core Innovation)

Cheap guard + lazy escalation + audit

④ Supporting Components

RAG tools, multi-agent, self-consistency

⑤ Demo & Conclusion

Task & Scoring Rule

Input: `task.json` + heterogeneous context

(CSV, SQLite, JSON, Markdown rules, record-text prose, images)

Output: `prediction.csv` scored by **column-content signature matching**

- Column names **ignored**; row order **ignored**
- Multi-column match via bipartite augmenting paths
- Normalize: numeric $\rightarrow 10^{-2}$, string `strip+lower`

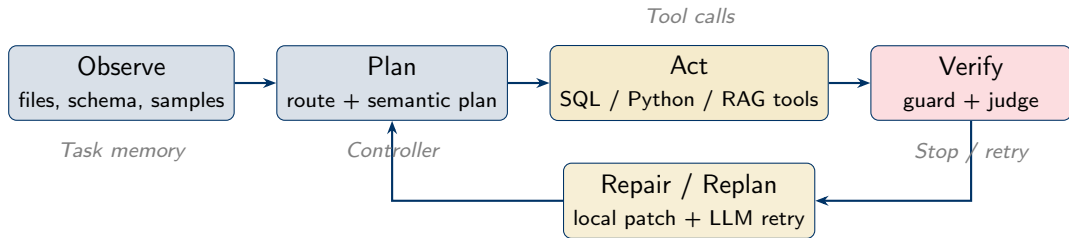
$$\text{score} = \max\left(0, \text{recall} - \lambda \cdot \frac{\text{extra_cols}}{\text{pred_cols}}\right), \quad \lambda = 0.5$$

Design implications

- **Fewer columns** — every extra wrong col is penalized
- **Preserve granularity** — `first_name+last_name` beats merged name
- **Match dtypes** — `163109.0` $\neq$ `163109`

Why Agentic? — Controller, Tools, Memory, Verifier

Raw ReAct is too unconstrained: easy tasks waste tokens on repeated tool calls; hard tasks drift across intermediate observations; final tables need strict shape control and semantic verification.



Design principle

The agent keeps a task trace, chooses tools, observes tool outputs, and only stops when the verifier accepts the answer. Simple tasks still pay minimal LLM cost.

LangChain vocabulary

- Agent = model + tools + iterative loop
- Runtime graph = model node, tool node, stop / retry edges
- Middleware controls context, tool access, guardrails
- LangSmith-style tracing is useful for debugging

Our implementation

- ModelAdapter = provider abstraction
- ToolRegistry = typed tool catalog
- ReActAgent = model-tool loop
- Orchestrator = planner-specialist-synthesizer DAG
- `trace.json` = memory + audit trail

Position

We use a lightweight LangGraph-style runtime instead of depending on LangChain directly. The architecture is compatible with a LangChain adapter: wrap ToolRegistry as LangChain tools, keep the same router, verifier, and answer validator.

Agent Controller — Dispatch by Shape, Not Difficulty

TaskCompiler (zero LLM):

- Per-file scan → SourceCapability
headers, dtypes, cardinality, samples
- Task-level aggregation → ExecutionProfile
 - context_size, source_shape
 - verifiability, operation_complexity
 - semantic_rule_required
- **Single source of truth** — reused by
prompt, static checker, guard, judge

Router rules (hard, deterministic):

- weak / image → multi_agent
- long_docs → RAG-enabled operator
- semantic_rule_required →
tool_first_mixed
- small+programmatic+low → easy

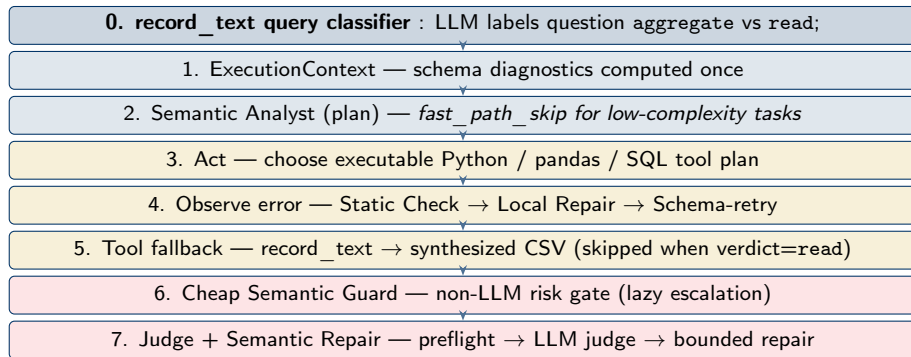
Cascade by *failure type*, not difficulty:

zero_rows → **no upgrade** (column wrong, not route)
retrieval_empty → RAG route
semantic_consistency_failed → tool_first_mixed
syntax/static/exec error → sibling operator

Cross-model verify (optional)

Primary + verifiers run in parallel; keep columns whose signatures agree $\geq$ min_agreement — directly targets the $-\lambda$ extra penalty.

Data Agent Controller — 8 Decision Points



Shared evidence — debug_steps

Every generated program emits `schema_inspection`, `used_columns`, `filter_conditions`, `join_keys`, `intermediate_counts`. Read by **3 consumers**: cheap guard, preflight, LLM judge — single audit trail, no drift.

Schema Grounding + Static Checker + Local Repair

Schema Grounding

Tabular-source prompt hint

- Real-column TOP-K hints
- **Per source-kind:**
- **CSV / SQLite** — full scoring
- **JSON** — sample + record fields
- **text / image** — skip

Static Checker

Pre-execution gate

- Python AST + SQL scan
- `no_such_column`,
`bad_join_key`, `_x/_y` suffixes
- Runs *before* execution
- Blocks predictable failures

Local Repair (0 LLM)

Deterministic patcher

- 10+ deterministic patches
- Fuzzy column-name fix
- Cast merge keys to string
- Append missing answer
- Zero-row filter normalize

Runtime safety net

Grounding is only a prompt hint. The real defense is runtime evidence: **schema_inspection** → **static checker** → **cheap guard** → **LLM judge**. All read the same `SourceCapability` + `trace.json`, so the audit trail does not drift.

Cheap Semantic Guard — Non-LLM Risk Gate

Position: codegen succeeded, answer structurally valid, $\rightarrow$ trust this answer?

$$\text{risk_score} = \min(1.0, \sum \text{unique_weights}), \quad \text{escalate} = \text{any error} \vee \text{score} \geq 0.5$$

Category	Risk Code	Wt	Meaning
Execution	execution_failed / empty_answer_rows	1.0 / 0.75	No/zero answer rows
Repair	suspicious_fallback:schema_retry	0.8	Needed schema retry
Trace	trace_missing:schema_inspection	0.8	No real-column check
Trace	zero_intermediate_count	0.8	Intermediate count drops to 0
Coverage	field_coverage_mismatch	0.75	Grounding candidate unused
Formula	formula_without_trigger	0.75	Unasked formula used

Killer example

Q: *"patients with severe degree of thrombosis"*. Code uses `disease=="thrombosis"` but grounding found `degree_of_thrombosis` $\rightarrow$ `field_coverage_mismatch` (0.75) $\rightarrow$ **escalate to judge** instead of shipping wrong answer.

Lazy Escalation + Analyst-Exception Recovery

1. Post-codegen gate

After codegen succeeds and the answer is structurally valid, the **Cheap Guard** scores 10+ deterministic rules (non-LLM). If score < 0.5 → **fast return**, zero extra LLM cost. Otherwise escalate.

2. Escalation flow (LLM path)

- **Analyst** (1 LLM call) — emits semantic plan
- **Judge** — compares plan vs code
 - pass → done
 - fail / low_confidence → **Semantic Repair** rewrites code
- Judge-Repair loops at most **3 rounds**

3. Analyst-exception recovery

stage=analyst_exception → retry once with force=True; still fails → low-confidence synthetic plan → **judge must run**.

4. Observability — final_gate

Top-level field in trace.json:

- plan+judge — strong path
- escalated+judge — recovered (retry / fallback / lazy escalation)
- cheap_guard_only — guard fired but judge did not run (investigate)
- bypassed — no semantic check

5. Cost

- Simple task: **1 LLM call** (codegen only)
- Rule-based task: 3–5 calls (analyst + codegen + judge ± repair)
- Analyst exception: +1 retry +1 judge (worst case)

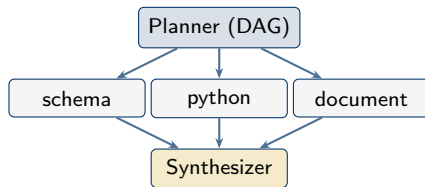
Core guarantee

No suspicious answer ships unchecked: if guard escalates, judge *will* run — even if analyst fails.

RAG (long_docs route):

- Heading-aware chunking (1200 chars, overlap 150)
- **Query expansion:** paraphrases + HyDE
- **Hybrid retrieval:** BM25 || Embedding
- **RRF fusion** (rank-only, no score calibration)
- Optional cross-encoder reranker

Multi-agent (extreme route):



Multi-Agent Mode — Roles & Permissions

1. Planner

- **Job:** decompose the question into a DAG of sub-tasks
- **Output:** JSON plan with dependencies
- **Tools:** none; pure reasoning only
- **Constraint:** cannot call answer or report

2. Specialists (5 kinds, parallel execution)

Kind	Allowed tools
Schema	<code>list_context</code> , <code>read_csv</code> , <code>read_json</code> , <code>read_doc</code> , <code>inspect_sqlite_schema</code>
SQL	<code>list_context</code> , <code>read_doc</code> , <code>inspect_sqlite_schema</code> , <code>execute_context_sql</code>
Python	<code>list_context</code> , <code>read_csv/json/doc</code> , <code>inspect_sqlite</code> , <code>execute_sql</code> , <code>execute_python</code>
Document	<code>list_context</code> , <code>read_doc</code> , <code>read_json</code> , <code>read_csv</code>
Generic	all tools; full ReAct

- Terminal tool = report, not answer
- `read_doc` capped at 2 calls per specialist
- Max 12 ReAct steps per specialist

3. Synthesizer

- **Job:** merge specialist findings into one final answer
- **Input:** all report payloads
- Only role allowed to call answer
- Max 6 ReAct steps
- Synthesis failure → one replan allowed

4. Orchestrator

- Dispatches sub-tasks via ThreadPool
- Same-layer specialists run **in parallel**
- Enforces DAG dependencies
- Layer N waits for layer $N - 1$
- Iterative refinement: 1 replan max
- Budget cap: `max_multiagent_fallbacks = 1`

Key design: permission isolation

Specialists cannot submit the final answer. They only report findings. The Synthesizer must cross-check all reports before committing the final answer.

Self-Consistency, Budget & Demo

Self-consistency — column-signature voting:

- N passes at $T=0.7$ with different seeds
- `target_count` = mode of column counts
- Top-`target_count` signatures by frequency = winning set
- Pick sample with max coverage, min extras
- **Directly targets** recall — $\lambda \cdot \text{extra}/\text{pred}$

Budget controller (per task):

- LLM / tool / time caps
- Local & reasoner repair caps
- Multi-agent fallback cap

Streamlit demo:

- Real-time subprocess log (ProgressLogger)
- **Reasoning timeline**: schema $\rightarrow$ route $\rightarrow$ codegen $\rightarrow$ guard $\rightarrow$ judge
- Final answer table + generated Python
- Full `trace.json` (collapsible)

```
uv run streamlit run \
  scripts/demo_app.py
```

Auditable, explainable, reproducible.

Summary & Future Work

What we built:

- Agentic controller with explicit tool actions
- Deterministic task profiling (zero LLM)
- Shape-based routing + failure-type cascade
- 10+ local repair rules (zero LLM)
- **Cheap Semantic Guard** — non-LLM risk gate
- Analyst-exception recovery + audit trail
- Production-grade RAG (BM25+Emb+RRF)
- Multi-agent DAG with bounded replan
- Column signature self-consistency

Observed performance (public):

Easy: ~70 Medium/Hard: ~60

Future work:

- Vision executor for image tasks
- Guard-weight auto-calibration from trace FP/FN
- Per-value majority voting within columns
- Smarter record-text extraction for long prose
- Submit to hidden test after tuning

Thank You

Questions?



香港科技大學(廣州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Code: github.com/zuelmaruixin/kdd